ASCE

BIM Library Transplant: Bridging Human Expertise and Artificial Intelligence for Customized Design Detailing

Suhyung Jang, S.M.ASCE[1]; and Ghang Lee, Ph.D.[2]

Abstract: This study introduces a framework for transplanting a building information modeling (BIM) library. Design detailing constitutes 50%–60% of the total design time, even within the BIM context. Previous studies have highlighted the potential of integrating BIM and artificial intelligence (AI) for enhanced productivity. However, challenges arise due to architects' preferences for unique project-specific details when applying generalized AI approaches based on big data. To address this, we propose a BIM library transplant framework. This framework automatically identifies objects at a high level of development (LOD) from a selected existing BIM model (i.e., a donor model) and matches them with low-LOD objects in a new model (i.e., a recipient model). Subsequently, it replaces the low-LOD objects with corresponding high-LOD objects. The framework involves three steps: (1) extracting the library from the donor model, (2) matching the library, and (3) transplanting the library from the donor to recipient model. To validate its efficacy, we implemented the BIM library transplant framework as a Revit add-on, employing the random forest classifier as the object-matching AI model. Our results indicate that the implemented framework has the potential to reduce detailing time by approximately 60%–70%, while achieving an accuracy of 65%–80%.
DOI: 10.1061/JCCEE5.CPENG-5680.

Author keywords: Building information modeling (BIM); Artificial intelligence (AI); Level of development (LOD); BIM authoring; Integrated human–machine intelligence (IHMI).

Introduction

Design detailing typically incurs 50%–60% of the total person-hours in an architectural design project (Eldin 1991), even when building information modeling (BIM) is adopted (Giel and Issa 2013; Czmoch and Pękala 2014; Koo et al. 2017). The integration of artificial intelligence (AI) and BIM is promising for achieving automated detailing, which allows architects to dedicate more time to creative pursuits (Pan and Zhang 2023). However, owing to the unique and subjective nature of architectural designs, such an integration is challenging, and makes these designs distinct from other domains. Although AI commonly exploits consistent patterns within standardized data sets across various domains, architectural designs resist such uniformity, and are shaped instead by distinct contexts and considerations. This differentiation stems from the following factors:

- Diversity: The inherent diversity in design projects originates from a multitude of elements, materials (Hosseinijou et al. 2014), systems (Yan et al. 2022), and project-specific variables such as regional climate (Phillipson et al. 2016), building occupancy (Zambrano et al. 2021), size, and other performance objectives (Lin et al. 2021). This diversity complicates the use of AI algorithms, which thrive on extensive, high-quality, and relatively homogeneous data sets (Mathew et al. 2021). Furthermore, transferring relevant knowledge from one project to another can be challenging owing to the varying characteristics of different projects.
- Subjectivity: Architectural design detailing is largely subjective and is influenced by the unique preferences, experiences, and design styles of architects (Senior et al. 2006; Lu and Liu 2011). The performance of AI models typically is evaluated using quantifiable data and objective rules (Davis 2019; Noguerol et al. 2019). However, the subjectivity of design makes it difficult to evaluate an AI model based on quantifiable data and objective rules.

The intricacies of these design characteristics pose challenges for traditional AI methods that rely on extensive data sets because these methods produce generalized outcomes that fail to capture architectural intricacies unique to each project. To address this concern, this study introduces a BIM library transplant framework, which seeks to automate design detailing by aligning with the common practice of reusing details from prior projects. The proposed framework automatically identifies and substitutes low-level BIM libraries in a new project's model by correlating them with high-level BIM libraries from a selected past project's model.

The remainder of this paper is structured into six sections. Section "Literature Review" provides a review of the literature pertinent to this study. Section "BIM Library Transplant" introduces the framework and its modules. Section "Implementation" details the current version of the framework used for validation. Section "Validation" formulates research questions and outlines the experimental design. Section "Results" evaluates the performance of the proposed framework, focusing on accuracy and efficiency. Section "Discussion" analyzes the results, identifies existing gaps, and suggests future research directions. Finally, section "Conclusion" provides a summary and final remarks.

[1]Ph.D. Student, Building Informatics Group, Dept. of Architecture and Architectural Engineering, Yonsei Univ., Seoul 03722, Korea. ORCID: https://orcid.org/0000-0001-8289-0657. Email: rgb000@yonsei.ac.kr

[2]Professor, Dept. of Architecture and Architectural Engineering, Yonsei Univ., Seoul 03722, Korea (corresponding author). ORCID: https://orcid.org/0000-0002-3522-2733. Email: glee@yonsei.ac.kr

Note. This manuscript was submitted on August 29, 2023; approved on December 4, 2023; published online on January 9, 2024. Discussion period open until June 9, 2024; separate discussions must be submitted for individual papers. This paper is part of the ***Journal of Computing in Civil Engineering***, © ASCE, ISSN 0887-3801.

Literature Review

The adoption of BIM has led to significant productivity enhancements in the architectural industry. A major factor contributing to the efficiency of BIM is parametric modeling, which is a technique that facilitates rapid modifications in design by focusing on specific attributes. This reduces the need for manual adjustments, which often are accompanied by traditional design changes. For example, Lee et al. (2006) demonstrated that integrating design and engineering knowledge into parametric models can enhance the representation of building objects and behaviors. Liu et al. (2018) proposed a rule-based approach to minimize material waste in light-frame residential buildings using parametric modeling. Hamidavi et al. (2020) proposed a procedure for linking architectural models with those of structural engineers to automate the generation of optimized structural designs based on architectural models.

A common approach to automated design detailing based on parametric models is the use of genetic algorithms. Mangal and Cheng (2018) proposed a BIM-based framework using a hybrid genetic algorithm to automate steel reinforcement optimization in RC frames. Similarly, Tafraout et al. (2019) proposed a method to automatically derive the optimal structure for a given architectural model using genetic algorithms. Wu et al. (2021) employed genetic algorithms to optimize floor tile layouts. Bourahla et al. (2022) developed a procedure to optimize shear wall panels for earthquake-resistant cold-formed steel structures using genetic algorithms.

Despite its advantages, parametric modeling presents challenges owing to the rigidity of predefined rules. Developing such rules can be laborious and time-consuming. This inherent rigidity can constrain the adaptability of the design detailing based on parametric modeling (Yenerim and Yan 2011). Although the parametric modeling approach is advantageous from the perspective of controlled parameters, design detailing often involves diverse and subjective factors that vary across projects.

AI-based approaches, particularly generative adversarial networks (GANs), have been adopted extensively for design detailing. Unlike parametric modeling–based approaches, GAN-based methods learn from training data to identify and replicate design patterns. Zhang et al. (2019) proposed an automatic generation method for mixed stylistic enhancements using a conditional Wasserstein GAN. Sun et al. (2022) developed a GAN-based tool for urban renovation that abstracts historic architectural styles and generates stylized facades. Luo and Huang (2022) introduced a generative adversarial model that combines vector generation and raster discrimination for floor plan generation. Similarly, Qian et al. (2022) used a self-sparse GAN to generate architectural shape sketches for early-stage design. Ghannad and Lee (2022) proposed a coupled GAN-based framework for automated modular housing design generation to facilitate the structural modularization of housing design layouts. Ye et al. (2022) developed a GAN-based model to generate renderings for urban master plans. However, GAN approaches prioritize visually represented information, and require substantial data sources for training. Their results often replicate existing design patterns, thus producing generalized outcomes and restricting user-engaged modifications.

Efforts also have been made to integrate AI with BIM for informed decision-making regarding design details. The predefined BIM object classes, which include physical and functional attributes of building objects, play a pivotal role. As emphasized by Leite et al. (2011), BIM objects with a high level of development (LOD) can promote informed decision-making throughout the design and construction phases.

Some researchers have focused on the retrieval of desired BIM objects to support architects' decision-making processes. Gao et al. (2015) developed a semantic search engine to retrieve BIM objects from online BIM libraries. Wu et al. (2019) proposed a natural language processing (NLP)-based method that leverages the geometric representation features of building objects to query them from BIM libraries. Li et al. (2020) proposed a similarity measurement method for the effective retrieval of BIM objects based on attribute information and Tversky similarity. Other studies have focused on classifying BIM objects. Koo et al. (2019) used one-class support vector machines to verify the integrity of BIM objects and detect misclassifications. Kim et al. (2019) proposed an approach to automatically classify BIM objects using convolution neural networks (CNNs).

Although the integration of BIM and AI aims to simplify informed decision-making, it does not necessarily alleviate the laboriousness of design detailing. Laboriousness arises from manually determining the specific details to be applied to each building object. In addition, the current big-data-driven approach is focused on providing generalized solutions; therefore, it often is limited to reflecting personalized preferences or varying project conditions.

To address the identified gaps and challenges, we propose a BIM library transplant framework. This framework leverages high-level BIM from past projects, thereby enabling the transfer of relevant and detailed information to lower-level BIM for new projects. This approach aims to offer a more customized and adaptive solution for automated design detailing, bridging the gap between AI and human expertise for enhanced automated design detailing.

BIM Library Transplant Framework

Fig. 1 explains the concepts and terminology used in this paper for comparison with those used in Revit, the tool that was used for implementation in this study. The concepts and terminology adhere to the object-oriented programming (OOP) concept. In the BIM library transplant framework, the highest object class is the abstract class, which cannot be instantiated. A subtype of an abstract class is a class, which defines the attributes and behaviors of an object. A subtype of a class is a subclass. An example of a class or a subclass placed in a BIM model is referred to as an object. A library is a collection of classes or subtypes. In Revit terminology, an abstract supertype is a category, a class is a family, a subclass is an (element) type, an object is an example or element, and an object identifier (ID) is an element ID.

Fig. 2 presents the BIM library transplant framework. The framework utilizes two input BIM models: the donor and recipient models. The donor model, which is derived from a previous project, contains high-level BIM objects, ranging from LOD 350 to 400. In contrast, the recipient model represents an initial BIM model for a new project, with BIM objects of about LOD 200. In the models, low-level BIM objects provide basic details such as location and size, whereas high-level objects offer intricate specifics such as operational features, attachment methods, and detailed structural components throughout the detailing process (Fig. 3).

The principal aim is to locate the requisite recipient model details within the donor model and seamlessly transplant these details into the recipient model, adapting them to the new design context. This concept relies on the notion that the BIM models' building object details (i.e., classes and subclasses) are integrated, predefined, or revised as referenced files, in which their collections commonly are known as BIM libraries. The iterative application of this process permits multiple donor models, handpicked by architects, to be integrated into a single recipient model.

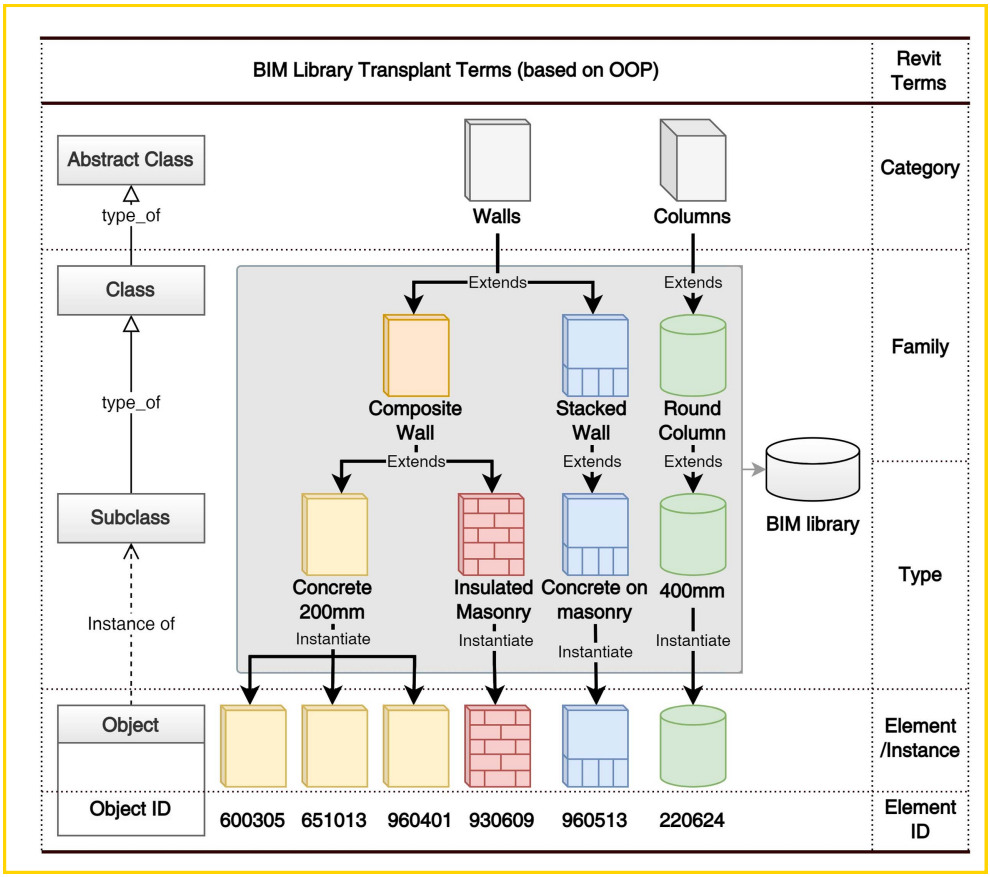

Fig. 1. BIM library transplant terminology and Revit equivalents.

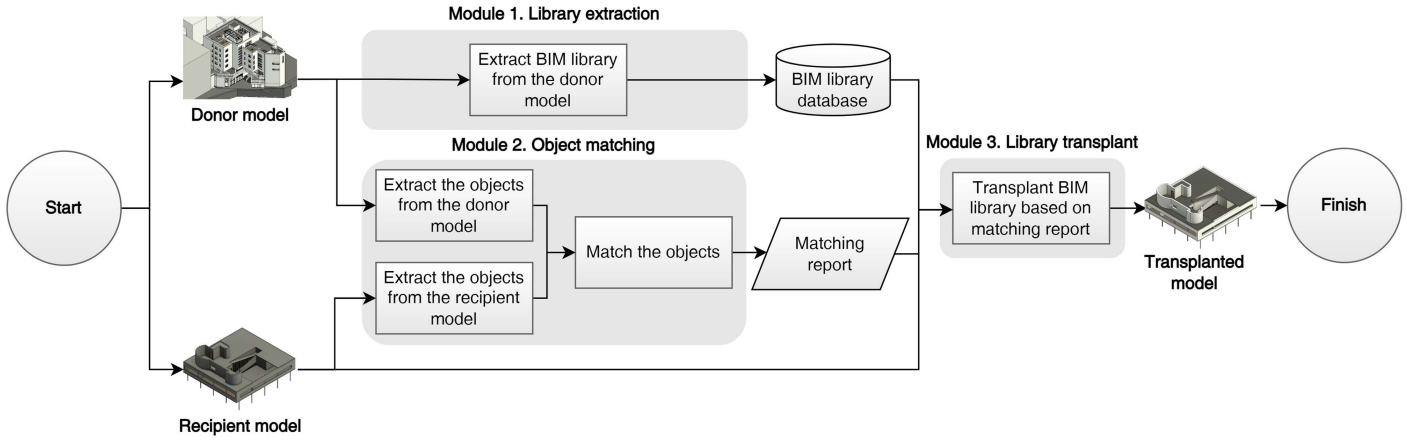

Fig. 2. BIM library transplant framework.

This framework encompasses three distinct modules: library extraction, object matching, and library transplant. The library extraction module extracts the classes utilized in the donor model, forming a BIM library database. The object matching module draws an analogy with the donor organ matching process in organ transplants, constituting the pivotal core of the library transplant framework. Within this module, a matching class from the library database is selected for each building object within the recipient model, and the outcomes are presented in a matching report. The library transplant module translates these findings into adjustments within the recipient model (i.e., the transplanted model), guided by the matching report.

The framework's objective is to empower architects with automated detailing capabilities, while concurrently tailoring detail selection to individual preferences and requirements. This approach fosters collaborative synergy between AI and human proficiency. The pivotal matching algorithm within the framework facilitates the extraction and adaptation of intricate domain insights with shared attributes spanning multiple projects. As a result, AI becomes adept at managing the inherent diversity across design projects, recognizing patterns, rules, and pertinent particulars among diverse projects. This process facilitates personalized architectural design detailing, drawing from knowledge and expertise accumulated from past endeavors. Moreover, the AI model underpinning the matching

J. Comput. Civ. Eng., 2024, 38(2): 04024004

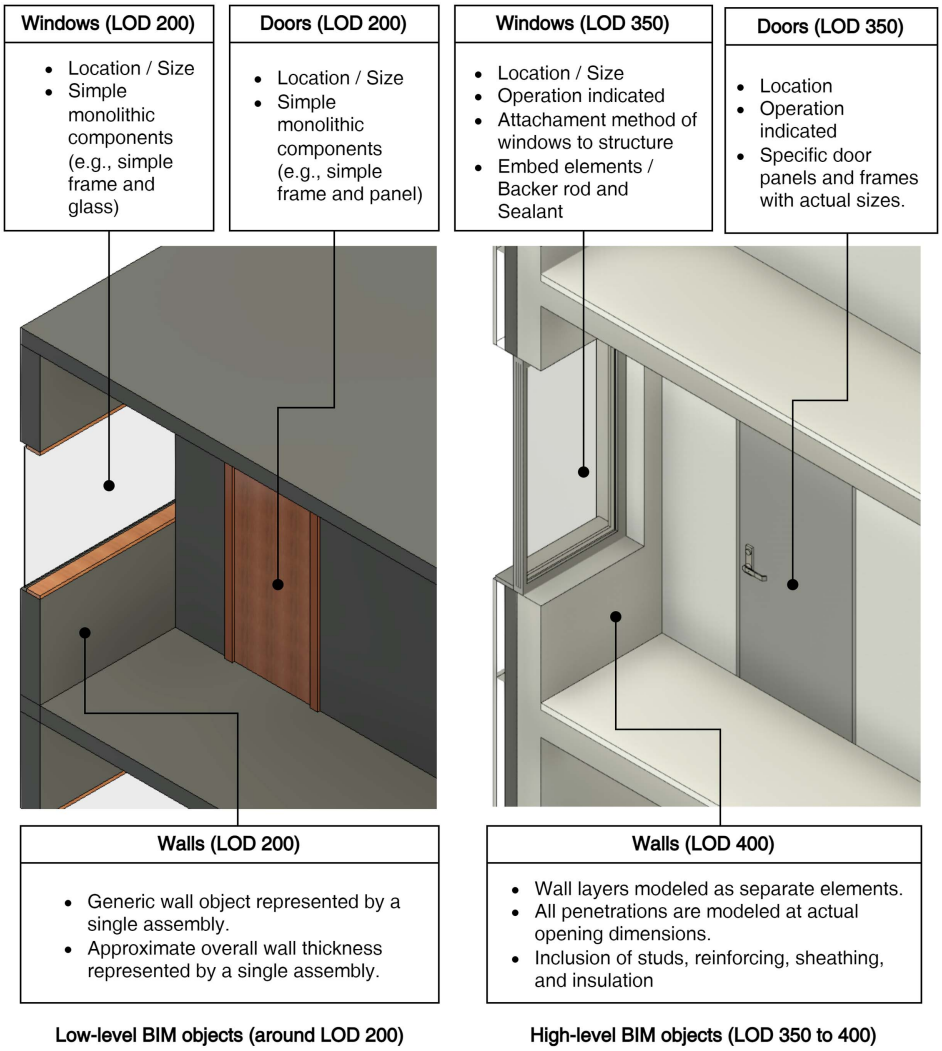

Fig. 3. BIM objects at different levels of development (LOD).

algorithm can be fine-tuned and updated as the system performance evolves.

Implementation

We developed an add-on for Autodesk Revit 2024 to implement the BIM library framework using the C# application programming interface (API) of Revit. The scope of implementation included handling six major BIM abstract classes: composite walls, curtain walls, floors, windows, doors, and columns.

Fig. 4 displays screenshots captured during the execution of the BIM library transplant add-on. The add-on can be run as an external command when a recipient model is open in Revit. During execution, the system prompts users to select a BIM model that can serve as the donor model. After the selection, the system manages the steps of library extraction, object matching, and library transplant, enriching the recipient model with classes from the donor model. The following subsections present the specifics of each module.

Library Extraction Module

The library extraction module retrieves the BIM libraries from the donor model and archives them in a BIM library database. The pseudocode of the extraction algorithm is shown in Fig. 5. Initially, the module identifies and extracts the classes associated with each abstract class from the donor model. Subsequently, these classes are extracted from the donor model into the BIM library database.

Object Matching Module

The object matching module identifies the most suitable classes for each object within the recipient model. To achieve this, a random forest classifier (RFC) was trained as a classification model. The training involved utilizing the attributes of the building objects sourced from the donor model as features, while assigning the classes as corresponding labels.

Random Forest Classifier

The critical characteristic of the object matching module is its ability to handle a diverse array of data types. This includes discrete, binary, and continuous data, because the attributes of BIM objects often involve multivariate data. These data combine contextual and geometric attributes, which are crucial for discerning the appropriate details for various building objects.

Kotsiantis et al. (2007) highlighted that among several machine learning classification techniques, decision tree–based methods are optimal for managing multivariate data types, including discrete, binary, and continuous data (Table 1). Neural networks and

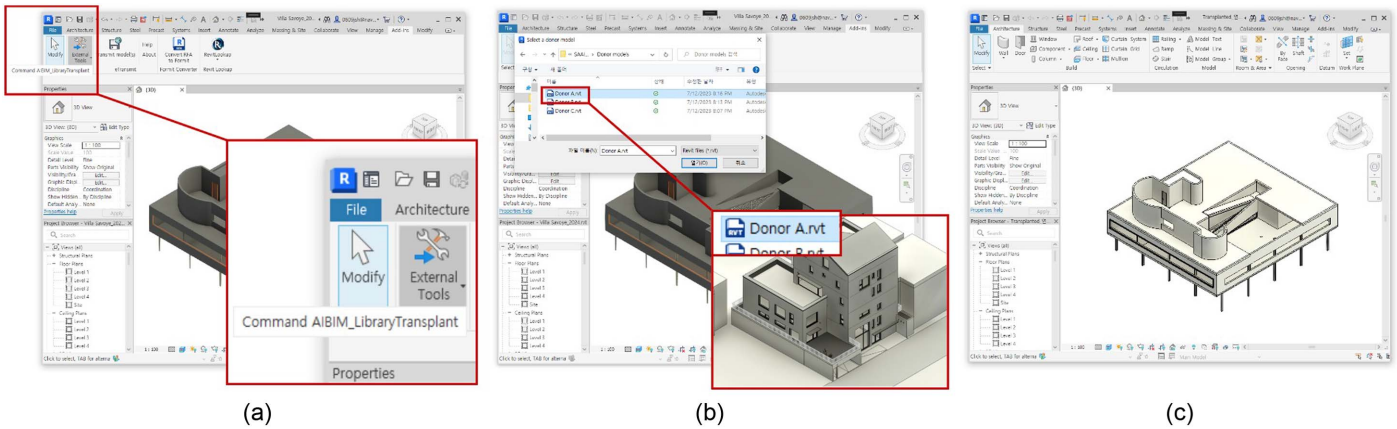

Fig. 4. Screenshots of the implemented BIM library transplant add-on: (a) execute the add-on with the recipient model opened; (b) select the donor model to extract BIM library; and (c) transplant the classes from the BIM library database to the recipient model.

Algorithm 1: Extraction algorithm

Input: Document *donorModel*
Output: FilteredObjectCollector *BIMLibraryDatabase*

```
Begin function LibraryExtraction
    For each abstractClass (i.e., composite wall, curtain wall, column, floor, window, door):
        Collect classes of the current abstractClass from donorModel using FilteredObjectCollector
        Copy the classes from the donor model to FilteredObjectCollector BIMLibraryDatabase
    End for
    Return BIMLibraryDatabase
End function
```

Fig. 5. Pseudocode of the extraction algorithm.

Table 1. Data types supported in each machine learning classifier

Data type	Decision trees	Neural networks	Naïve Bayes	*k*-nearest neighbors	Support vector machine
Discrete data	Y	—	Y	—	—
Binary data	Y	Y	Y	Y	Y
Continuous data	Y	Y	—	Y	Y

Source: Adapted from Kotsiantis et al. (2007).
Note: Y indicates that the data type is supported without preprocessing.

support vector machines necessitate preprocessing, such as one-hot encoding, for discrete features. The *k*-nearest neighbors method relies on distance computation, rendering it unsuitable for direct application with discrete features. Although naïve Bayes directly calculates probabilities from frequency counts in training data sets, it mandates converting continuous features into categorical features, rendering it inapplicable for continuous features without auxiliary techniques.

Among decision tree–based approaches, the RFC has proven its performance in classification tasks within the construction industry (Hoang and Nguyen 2019; Chun et al. 2020; Hoang and Tran 2023). RFC is an ensemble learning method that constructs multiple decision trees during the training phase. The data set, $D = \{(X_1, Y_1), (X_2, Y_2), \ldots, (X_n, Y_n)\}$, contains the feature vectors X_i for the ith instance and their corresponding labels Y_i. The RFC algorithm operates in three steps:

- Bootstrapping: This technique involves resampling the original data set D with replacement to produce n subsets, denoted B_i. Each B_i has the same size as D, but because of random sampling, some instances may appear multiple times, whereas others may not appear at all.
- Decision tree generation: A decision tree T_i is generated for each bootstrap sample B_i. At each node of T_i, m features are randomly selected from all the features, and the best split point among these m features is chosen to split the node. The parameter m remains constant throughout the forest growth.
- Voting: After all the decision trees $\{T_1, T_2, \ldots, T_n\}$ have been constructed, a new instance is introduced. Each tree T_i provides a classification, and the votes for each class are counted across all the trees. The class that receives the majority of votes is the prediction for a given instance. Mathematically, for a new instance with a feature vector X, the predicted class $\hat{Y}$ is determined by

$$\hat{Y} = \text{mode}(T_1(X), T_2(X), \ldots, T_n(X)) \quad (1)$$

Employing multiple decision trees and the subsequent averaging process markedly mitigates the risk of overfitting, which is a

common concern with solitary decision trees (Ho 1995). This characteristic aligns well with our scenario given the intrinsic imbalances among building objects across BIM abstract classes. Moreover, RFC excels in handling nonlinear classification tasks, effectively addressing the inherent nonlinear attributes found in BIM object features. Additionally, RFC exhibits resilience against outliers, which is a vital trait when confronted with BIM objects showcasing notable variations, even within the same BIM classes. Therefore, owing to its comprehensive nature, robustness, and adaptability, RFC was identified as the optimal matching algorithm for our study.

Features

To train the RFC, the object matching module first extracts two categories of features, namely contextual and geometric properties, from the building objects of the donor model.

Contextual properties encompass an object's placement and relationships within its broader project environment, delineating the object's position and context within the structure. This entails attributes such as the element's vertical position (base or top level), its connections or divisions within spaces (i.e., adjacent rooms), and its specific level assignment (level). Contextual properties illuminate an architectural element's situational and environmental relevance, offering insights into its functional role within architectural design.

Geometric properties emphasize the physical attributes of an architectural element, distinct from its context. These include the shape, size, and other attributes with measurable values such as sill height, perimeter, base offset, top offset, area, and slope. These properties provide unique insights into an object's role within the project. For example, wall objects that are used to represent external walls surrounding the external edges of the system are likely to be modeled with a longer perimeter than those representing internal walls, which divide indoor spaces. Consequently, these geometric properties also are utilized as features to determine the specific details to be applied.

Table 2 outlines the contextual and geometric properties of the six objects used as features to train the matching model. These properties are described in detail as follows:

- Composite and curtain walls: From the walls, the extracted features comprise contextual properties such as the base level, top level, and adjacent rooms, alongside geometric properties encompassing the height, length, area, base offset, and top offset. The base and top level properties serve to locate walls vertically between floors, establishing their vertical arrangement, whereas adjacent room properties indicate the walls' functional role in the building's horizontal layout. The physical dimensions of walls include height, length, and area. In addition, the base offset and top offset properties offer insights into positional deviations from the base or top levels, particularly pertinent in intricate designs.
- Columns: From the columns, the extracted features encompass the base level, top level, width, depth, base offset, and top offset. The properties of base and top levels provide understanding of the vertical positioning of columns within the building framework. The width and depth yield measurements of column dimensions, whereas the base and top offset properties supply details concerning potential positional deviations from the base or top levels, which are crucial for maintaining structural stability.
- Floors: Features derived from the floors encompass the level, perimeter, area, height offset, and slope. The level property indicates the floor positioning within the building's structure. Perimeter and area characterize the floor dimensions, whereas the height offset and slope detail deviations from the standard level and deliberate slope adjustments, which potentially are used for drainage or other functional considerations.
- Windows and doors: Windows and doors yield features such as level, height, width, and area. The level property aids in ascertaining the vertical positions of windows and doors within the structure. Height, width, and area furnish precise size measurements, which are pivotal for elements such as light ingress, ventilation, and aesthetic considerations for windows; whereas considerations for doors encompass accessibility and fluid movement between spaces. For windows, the sill height also is extracted, which is crucial for placement in relation to functionality.

By meticulously accounting for these specific contextual and geometric attributes, the trained RFC can be equipped to accurately categorize pertinent details from the donor model for application to each building object within the recipient model. However, the properties that are suitable for the matching algorithm are not confined to those employed in the current version of the implementation.

Algorithm

The RFC model was developed first in a Python environment using the scikit-learn library, and then integrated with the .NET

Table 2. Properties used as features for training by each abstract class

Abstract class	Composite wall	Curtain wall	Column	Floor	Window	Door
			Contextual properties			
Base level	Y	Y	Y	—	—	—
Top level	Y	Y	Y	—	—	—
Level	—	—	—	Y	Y	Y
Adjacent rooms	Y	Y	—	—	—	—
			Geometric properties			
Height	Y	Y	—	—	Y	Y
Sill height	—	—	—	—	Y	—
Length	Y	Y	—	—	—	—
Perimeter	—	—	—	Y	—	—
Width	—	—	Y	—	Y	Y
Depth	—	—	Y	—	—	—
Area	Y	Y	—	Y	Y	Y
Base offset	Y	Y	Y	Y	—	—
Top offset	Y	Y	Y	—	—	—
Slope	—	—	—	Y	—	—

Note: Y indicates that the property was used.

J. Comput. Civ. Eng., 2024, 38(2): 04024004

Algorithm 2: Matching algorithm

Input: Document *recipientModel*, Document *donorModel*
Output: Dataframe *matchingReport*

```
Begin function ObjectMatching
    For each abstractClass (i.e., composite wall, curtain wall, column, floor, window, door):
        Extract the feature values of all objects in the abstractClass from the donorModel as a
    Dataframe donor_{abstractClass}
        Extract the objectId and feature values of all objects in the abstractClass from the
    recipientModel as a Dataframe recipient_{abstractClass}
        Preprocess the donor_{abstractClass} and recipient_{abstractClass} encode categorical
    variables, handle missing or infinite values, and standardize the features
        Initialize the LabelEncoder and fit on all possible classes in donorModel
        Create a new instance of the RandomForestClassifier RFC
        Train the RFC on the donor_{abstractClass}
        Use the trained RFC to predict classes for the recipient_{abstractClass}
        Decode the predicted classes to their original form
        Return the objectId, abstractClass, and predictedClass to the recipientModel and save it as a
    Dataframe matchingReport_{abstractClass}
    Concatenate the matchingReport_{of all object abstractClass} as Dataframe matchingReport
    Return matchingReport
End function
```

Fig. 6. Pseudocode of the matching algorithm.

framework of the Revit C# API. The pseudocode for the object matching module is shown in Fig. 6. The algorithm uses the RFC to match appropriate classes for transplantation into building objects in the recipient model based on their usage in the donor model. Notably, the object matching modules are trained dynamically with each new donor model, utilizing it as a ground-truth data set, to better capture the features of the building objects contained within the donor model.

For each object classification process, the algorithm iterates through each object, extracting features and classes from both the donor and recipient models, along with the object's identification (ID) in Revit for the recipient model. These features then undergo preprocessing, which includes tasks such as encoding categorical variables, addressing missing or infinite values, and standardizing the data using a standard scaler. Such preprocessing ensures that the data are primed for the RFC.

Using the feature values from the donor model's objects and classes, the RFC is trained to discern the relationship between building object features and donor model classes. When it is trained, the RFC predicts the adequate classes for objects in the recipient model. This prediction is based on the unique features of each object, and the corresponding class names. The results of the prediction are compiled into a matching report, detailing the object ID, abstract class, and predicted class.

Library Transplant Module

The library transplant module involves transplanting equal classes within the BIM library database from the library extraction module based on the matching report, which is obtained from the object matching module. The pseudocode of the transplant algorithm is shown in Fig. 7.

The algorithm was devised to interpret object IDs, abstract classes, and predicted classes from the matching report using the switch-case clause to manage objects within the BIM library database that is categorized by abstract class. For each object ID, the algorithm identifies the corresponding BIM object and transforms the class into the predicted class. When windows, doors, and columns possess distinct size attributes such as width, depth, and height, the transplant algorithm generates a new subclass incorporating comprehensive object properties. Subsequently, the recipient model is enriched with details transferred from the donor model (i.e., the transplanted model).

Validation

To evaluate the implemented BIM library transplant framework, an experiment (Fig. 8) was designed to address the following research questions:

1. To what extent does BIM library transplant accurately represent an architect's design detail preferences?
2. How does the efficiency of using BIM library transplants compare with traditional design detailing?

First, the extent to which the implemented BIM library transplant accurately represents the architect's design details preferences was assessed through a comparative evaluation involving (1) the transplanted model—achieved by detailing the recipient model using the framework with a donor model; and (2) the architect-verified transplanted model—in which manual verification and revision by an architect were performed. The architect-verified transplanted model was employed as the benchmark for accuracy assessment. This involved extracting object IDs and classes from both the transplanted model and architect-verified transplanted models, followed by a comparison to gauge congruence between the two models.

Second, enhanced efficiency was gauged by quantifying the reduction in working hours required for manual design detailing. This involved a comparison of the time invested in design detailing when the implemented framework was employed in conjunction with manual detailing, as opposed to solely manual detailing. For benchmarking, the architect detailed the recipient model using classes from their personal donor model. Working hours for both manual detailing procedures were computed by subtracting the start time from the end time for each process, which were recorded using the advanced BIM logger developed at Yonsei University (Jang and Lee 2023). Enhanced efficiency was calculated by first determining the variance in working hours for generating the architect-detailed recipient model and the architect-verified transplanted model. This variance then was divided by the number of working hours required to produce a detailed recipient model.

Algorithm 3: Transplant algorithm

Input: Document *recipientModel*, FilteredObjectCollector *BIMLibraryDatabase*, Dataframe *matchingReport*
Output: Document *transplantedModel*

```
Begin function LibraryTransplant
  Declare a list for each: objectIds, abstractClasses, predictedClasses from matchingReport
  For each index in objectIds list:
    Declare objectId with the corresponding index from objectIds list
    Declare abstractClass with the corresponding index from abstractClasses list
    Declare predictedClass with the corresponding index from predictedClasses list
    Switch on abstractClass:
      Case "composite wall", "curtain wall", or "floor":
          Collect object of the objectId in the recipientModel
          Collect the correspondingClass of the predictedClass in the BIMLibraryDatabase
          Change the class of the object to the correspondingClass
      Case "window" or "door":
          Collect object of the objectId in the recipientModel
          Get the width and height properties of the object
          Create a string size from these properties
          If the size is unique,
            Add it to the HashSet and add a new subclass with these properties
            Change the class of the object to the new subclass
          Else,
            Collect the correspondingClass with these properties from the BIMLibraryDatabase
            Change the class of the object to the correspondingClass
      Case "column":
          Collect object of the objectId in the recipientModel
          Get the width and depth properties of the object with respect to various conditions
          Create a string size from these properties
          If the size is unique,
            Add it to the HashSet and add a new subclass with these properties
            Change the class of the object to the new subclass
          Else,
            Collect the correspondingClass with these properties from the BIMLibraryDatabase
            Change the class of the object to the correspondingClass
  Return the changed recipientModel as transplantedModel
End function
```

Fig. 7. Pseudocode of the transplant algorithm.

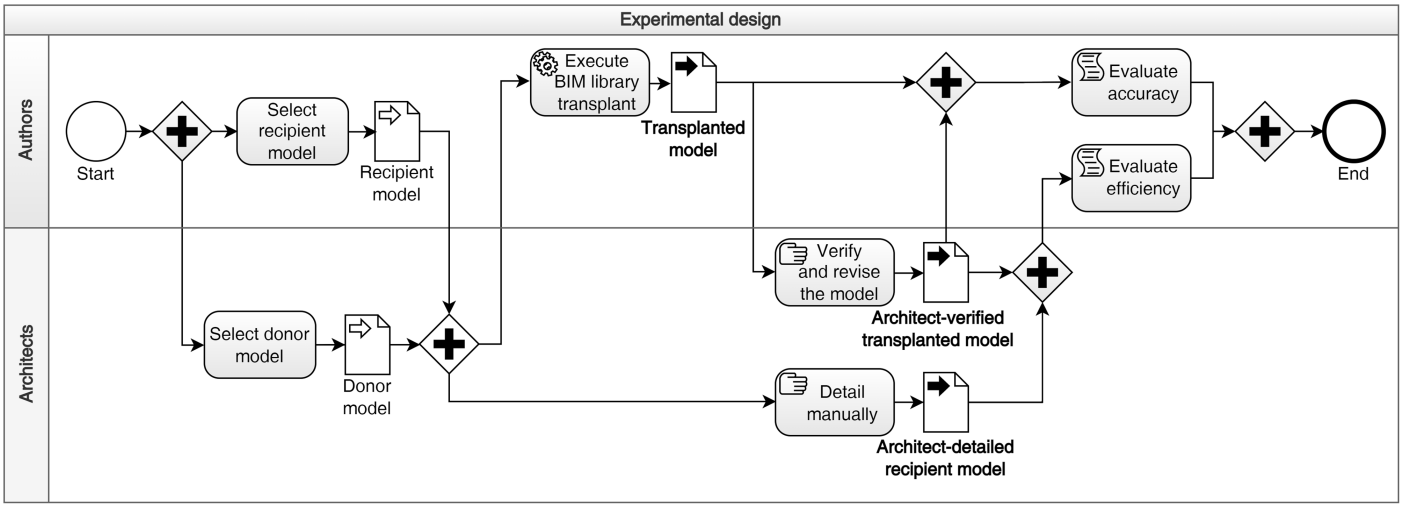

Fig. 8. Experimental design business process modeling and notation (BPMN).

The resulting ratio, multiplied by 100, yielded the percentage enhancement in efficiency.

The accuracy and efficiency of the proposed framework were assessed across pairings of a recipient model with three distinct donor models. In each pairing, the same recipient model was matched with a distinct donor model provided by registered architects participating in the experiment.

Participants

Participants were enlisted via a public announcement made within a Seoul-based architectural firm in South Korea. Eligibility criteria included (1) possessing over 1 year of BIM experience, (2) involvement in at least three BIM-centered projects, (3) capability to offer prior projects as donor models, and (4) access to Autodesk Revit 2024.

From the pool of prospective participants, two individuals fulfilled the specified criteria. One participant contributed a single project, whereas the other provided two projects, amounting to three donor projects available for validation. Prior to the experiments, the participants received a comprehensive briefing on the experimental setup to ensure smooth execution. Their familiarity with the donor models stemming from their involvement in those projects facilitated their understanding of the encapsulated details.

Recipient and Donor Models

The recipient model was a low-level BIM representation of Villa Savoye, a creation by Le Corbusier, fashioned using default generic classes and possessing LOD 200. Three high-level BIM donor models, each exhibiting LOD 350 to 400, were employed. These models represented diverse built projects in terms of size and occupancy. Donor A was a four-story residential building with a basement and gross floor area (GFA) of approximately 1,000 m^2. Donor B was a five-story residential building with two basement floors, featuring a GFA of approximately 4,000 m^2. Donor C was a 14-story mixed-use building with 3 basement floors and a GFA of approximately 6,000 m^2.

Fig. 9(a) shows orthographic views of the recipient and donor models. Figs. 9(b and c) illustrate the distribution of objects and classes by abstract class within each model. The recipient model exhibited the least number of objects and employed a solitary class per abstract class. Notably, despite its comparatively smaller gross floor area when measured against other donor models, Donor A encompassed a notably diverse array of classes, particularly concerning composite walls, floors, and doors. This diversity hints at the nuanced detailing exhibited by Donor A, specifically in the cited abstract classes. In contrast, although donor B comprised a greater quantity of objects, it stood out owing to its extensive application of curtain walls and reduced use of windows. Manual investigation determined that a significant portion of windows was configured employing curtain wall classes instead of window

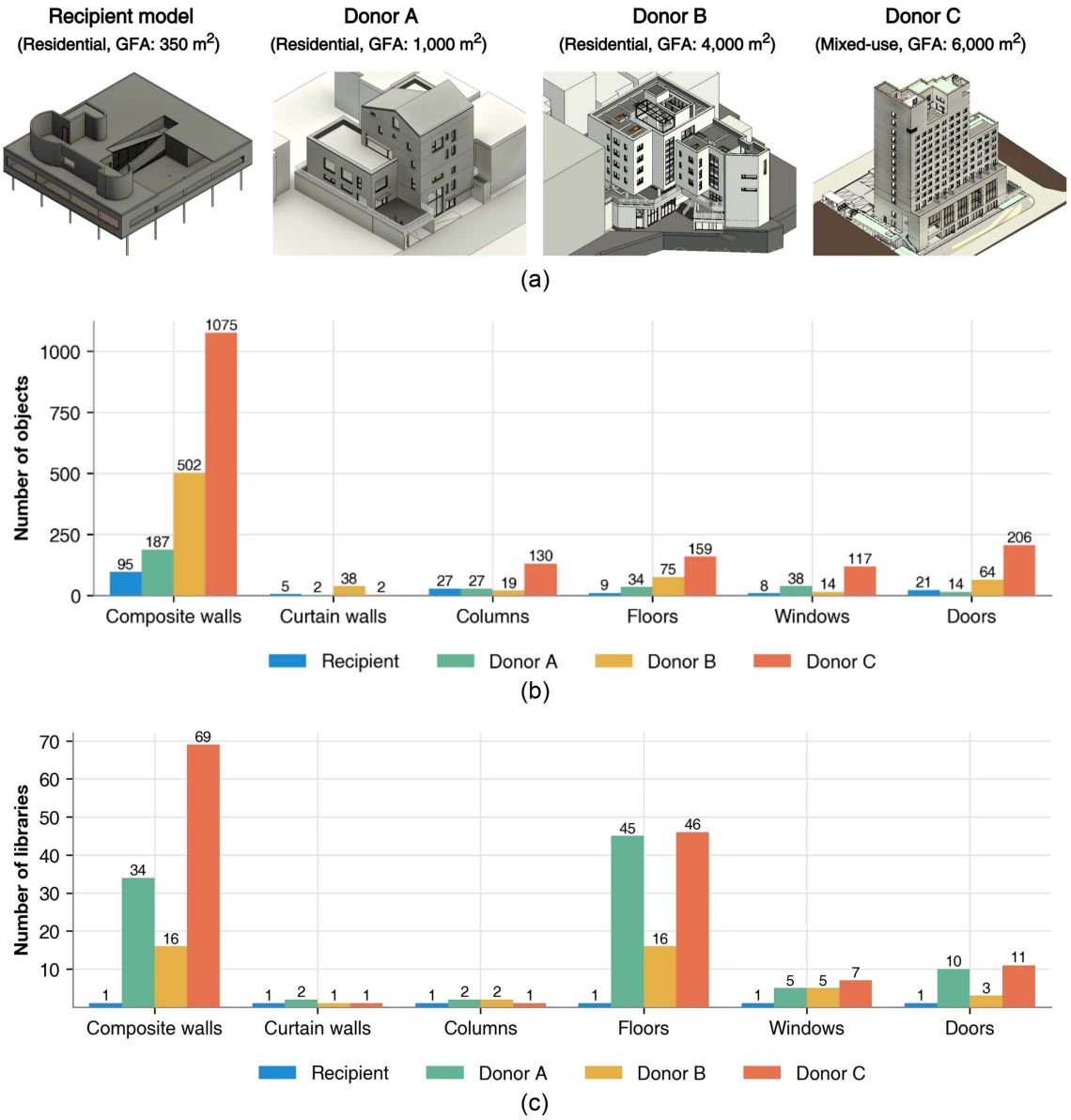

Fig. 9. Distributions of objects and classes within the recipient and donor models: (a) orthographic views; (b) distribution of objects by abstract class within each model; and (c) distribution of classes by abstract class within each model.

J. Comput. Civ. Eng., 2024, 38(2): 04024004

classes, underscoring distinct architectural detailing preferences among architects. Finally, Donor C, the sprawling mixed-use project, had the highest object count and significantly increased class application for composite walls compared with the other donors, predominantly owing to the increase in the number of adjacent rooms.

In summary, the combined analysis of recipient and donor models revealed distinctive architectural design and detailing styles. Each model, characterized by its distinct object and class allocation, presented a unique portrayal of architectural intricacy. The contrast in their detailed complexity underscores the diversity inherent in data sets, rendering them apt for validating the efficacy of the implemented BIM library transplant. The data sets, comprising the features and classes of building objects from the recipient Donor A, B, and C models used for training and validating the framework, are presented in Tables S1–S24.

Results

Increased Efficiency

Fig. 10 provides a comparative examination of the time consumed during manual design detailing, comparing scenarios with and without the use of the proposed BIM library transplant framework. The data set pertains to three unique projects identified as Donors A, B, and C. Each project is represented by a pair of bars, one denoting the timeframe for manual design detailing with the framework, and the other without it. Durations are expressed in minutes and seconds. The graph distinctly highlights the noteworthy reduction in design detailing time brought about by the incorporation of the BIM library transplant framework for all three projects.

For example, in the case of Donor A, the implementation of the BIM library transplant technique led to a notable efficiency increase of 61.96%. This translated into a reduction of detailing time from 9 min 41 s to 3 min 41 s. Parallel advancements were evident in the examples of Donors B and C, showcasing efficiency gains of 73.80% and 61.91%, respectively. These outcomes indicate that the BIM library transplant framework has the potential to markedly enhance the efficiency of the design detailing process. Consequently, it presents a promising avenue for streamlining architectural design detailing and elevating overall productivity.

Accuracy

The accuracy of the implemented BIM library transplant framework was evaluated by comparing the transplanted model with an architect-verified transplanted model. Fig. 11 provides a bar chart comparison of the framework accuracy for each abstract class across the different donor projects.

The framework achieved weighted accuracies of 77.09%, 83.93%, and 65.18% for Donors A, B, and C, respectively. The highest accuracy was attained with Donor B, which also displayed the least variability. However, accuracy varied across abstract

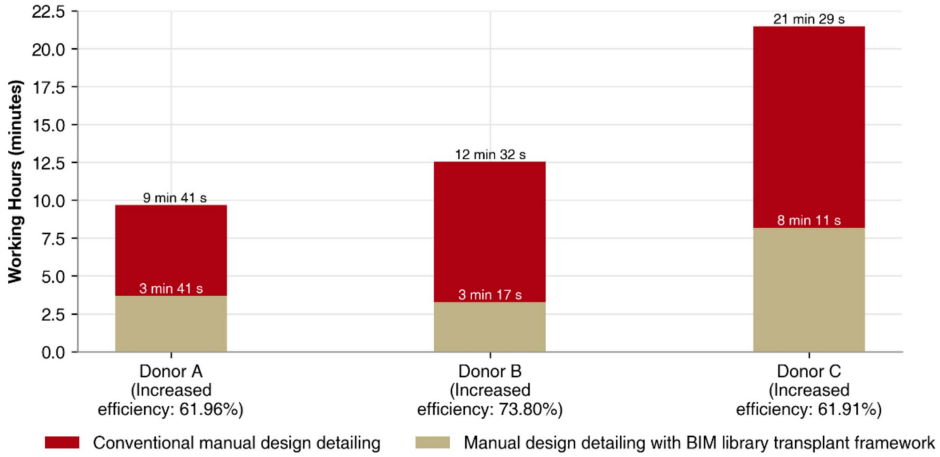

Fig. 10. Efficiency increase using the implemented BIM library transplant for design detailing.

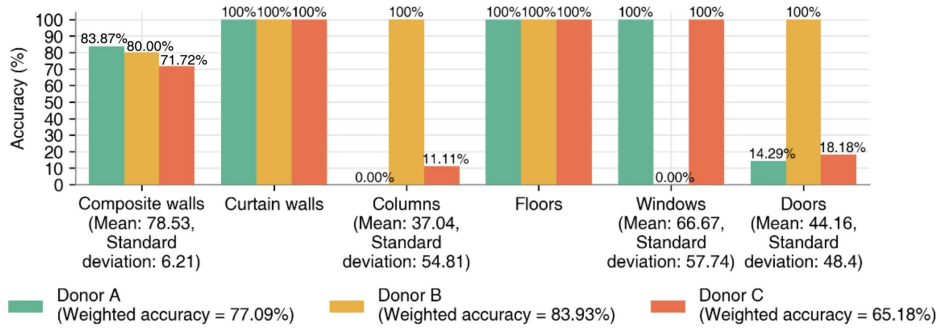

Fig. 11. Accuracy of the implemented BIM library transplant.

classes. Furthermore, the framework achieved 100% accuracy in identifying curtain walls and floor objects. In contrast, for composite walls, accuracies ranged between 71.72% and 83.87%, demonstrating a decreasing trend as the donor project size increased. A relatively modest standard deviation of 6.21 was noted, unlike for other abstract classes such as columns, windows, and doors, which exhibited significant performance variation. Mean accuracies of 37.04%, 66.67%, and 44.16%, respectively, were observed for these abstract classes, with corresponding standard deviations of 54.81, 57.75, and 48.4.

These results underscore the BIM library transplant framework's performance sensitivity to donor project selection and abstract classes. Although further validation is necessary, it can be inferred that similarities in project size and building occupancy might impact framework accuracy. Conversely, the framework demonstrates varying accuracy levels across distinct abstract classes. Whereas it excels for walls, curtain walls, and floors, it fluctuates considerably for other abstract classes such as columns, windows, and doors. The performance variation among different donor projects further emphasizes their role in influencing framework accuracy.

Discussion

This section provides a comprehensive analysis of the validation results, underscoring the current challenges of the implemented version and outlining potential avenues for future enhancements.

Current Challenges

The main challenges of the current version of implementation were identified as (1) selecting the appropriate donor model; (2) managing detailed building objects in donor models; and (3) addressing incomplete parameter application. The first two challenges are associated predominantly with the limitations in the matching algorithm, whereas the third pertains to both the extraction and transplant algorithms.

Selecting the Appropriate Donor Model

The success of the BIM library transplant is closely related to the selection of the donor model. For example, Fig. 12 illustrates H-section steel columns transplanted from Donor model A to the recipient model. The primary structural system of Donor A was a RC bearing wall system that transfers load through walls and does not use concrete columns. Instead, Donor A employed H-section steel columns for the low-rise section in front of the main building. As a result, the framework transplanted H-section steel column details into the recipient model instead of those of RC columns. However, the recipient model was intended to be a RC structural system, leading architects to manually revise them. This revision may not have been necessary if the architect had selected an appropriate donor model for the receiver model in the first place. Conversely, this result also highlights the need for a function that allows an architect to filter and transplant object details of their choice.

Detailing Building Objects in Context

Architectural details vary by context. Details for bathroom walls that require waterproofing usually differ from those for other wall types. Nevertheless, the representation of such a waterproofing layer can vary depending on the architect's modeling approach. For example, in Donor C, the tile and the waterproofing layers of its bathroom walls were modeled separately from the structural wall object (Fig. 13). Subsequently, a generic structural wall, designated WALL-200mm, occupied the most extensive area in Donor C (Table 3). However, such represented details also can be modeled as a single composite wall object.

The currently implemented matching algorithm does not choose details considering such context or the functional requirements of the objects. Furthermore, gaps, joints, and connections resulting from object replacements also are matters for future consideration. To address these challenges, refining the matching algorithm is pivotal, emphasizing a deep understanding of the architectural context and geometric characteristics of building objects within donor models.

Addressing Incomplete Parameter Application

Specific challenges, related to incomplete parameter applications, manifest when objects with unique geometric parameters demand intricate depictions. Fig. 14 shows curtain walls in the transplanted model and the architect-verified transplanted model, both utilizing Donor B. The curtain wall class in Donor B stipulated a fixed spacing for the vertical grid layout. Although the implemented framework correctly identified class usage, it did not apply layout rules, resulting in curtain walls without division. This hindered the creation of the intended curved landscape, resulting in a straight line. To overcome this, we propose refining the extraction algorithm to query vital properties determining parametric shapes and adjusting the transplant algorithm to accommodate selected classes more flexibly.

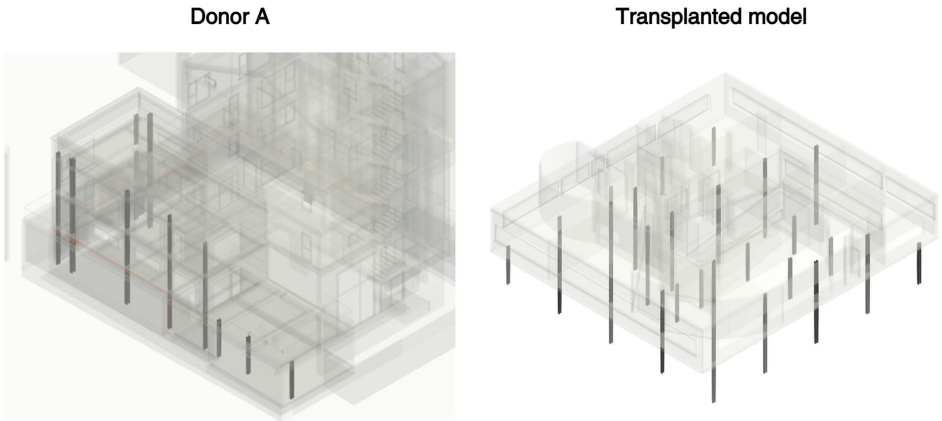

Fig. 12. Columns in Donor A and the transplanted recipient model.

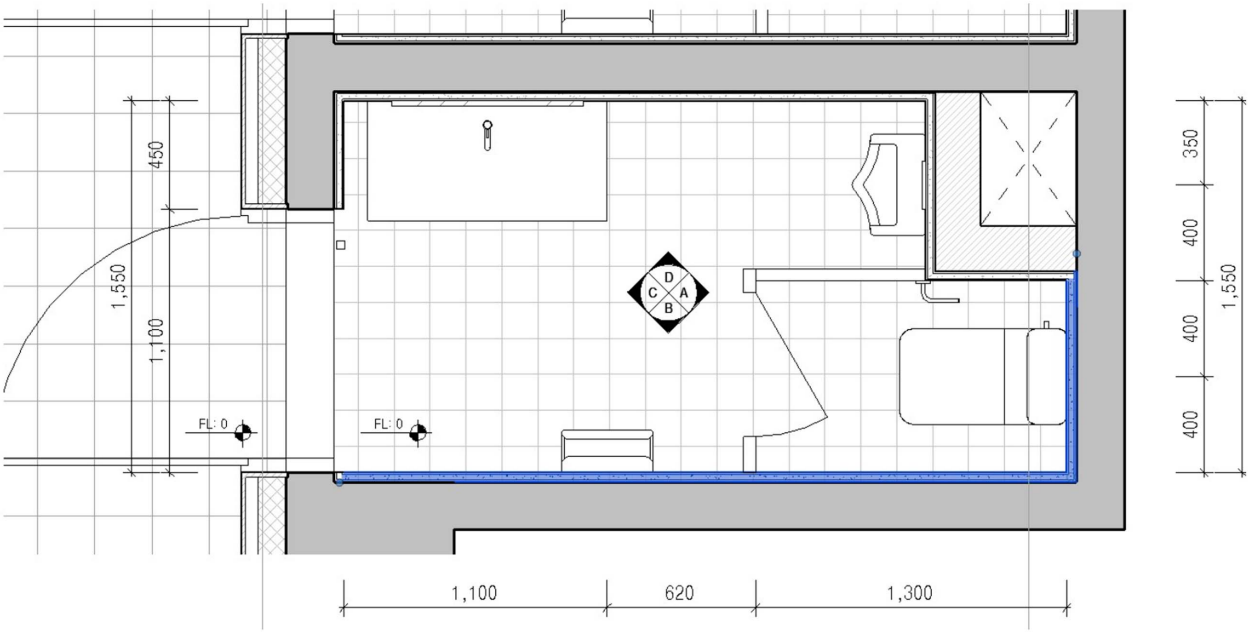

Fig. 13. Illustration of wall objects detailed with multiple layers in Donor C.

Table 3. Top wall libraries by total area used in Donor C

Library name	Total area (m^2)
WALL-200mm	4,383.14
C200/IN120/S50/T20	4,361.53
Ceramic tile/cement 20/waterproofing/cement brick 0.5B/cement 20/wallpaper	752.57
Cement brick 0.5B/waterproofing/cement 20/T9 ceramic tile	603.34
Waterproofing/cement mortar/ceramic tile	334.15
W/C400/IN40/A20/B90/M20/P	329.43
W/C300/IN40/A20/B90/M20/P	277.51
WALL-100mm	186.54
W/C600	176.12

Future Improvements

Given the aforementioned challenges, the current implementation of our proposed system still is in its preliminary stages. Building upon these identified challenges, the following paths for refinement emerge:

- Strategic donor model selection: Future research should explore the development of an automated system to evaluate and rank potential donor models, assisting architects in their selection process. Although the current framework is designed to operate primarily with a single donor model, it can accommodate multiple models through an iterative approach; however, this method may introduce potential conflicts. Comprehensive support for seamlessly integrating multiple donor models, thereby

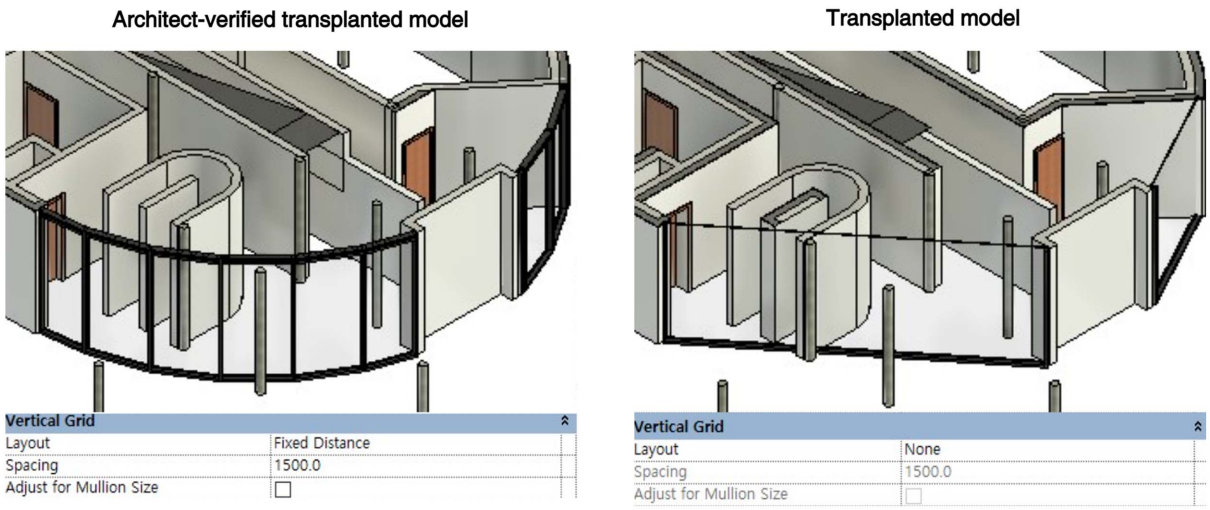

Fig. 14. Curtain walls in the architect-verified transplanted model and transplanted model for Donor B.

providing users with greater flexibility, has been identified as an area for future development.

- Enhanced matching algorithm: There is a pressing need to evolve the algorithm to discern more effectively when multiple BIM objects signify a single composite object. Gaining a deeper understanding of the architectural context, such as room types, functional areas, or adjacent BIM objects, can enhance the algorithm's accuracy in matching objects between donor and recipient models. This also could streamline the process, reducing the time and effort needed to identify and rectify design discrepancies.
- Parameter application refinement: Addressing the challenges related to incomplete parameter applications also is crucial. Developing an algorithm that can dynamically apply design rules based on the donor class's specifications may provide a solution that can ensure that design details are transferred correctly during the library transplant process. Moreover, developing an interactive interface can be beneficial, allowing users to adjust certain properties before finalizing the library transplant, and giving architects more direct control over the outcome.

In essence, these improvements aim to render the BIM library transplant framework more intuitive, accurate, and user-centric, ensuring that the resultant models satisfy the architects' design intent and standards. The integration of advanced algorithms for each module, coupled with user feedback, can pave the way for a more robust and efficient BIM transplant framework in the future.

Conclusion

This study introduces a BIM library transplant framework that utilizes existing BIM resources to automate new project detailing. This method caters to the diverse characteristics of architectural projects and respects individual architectural preferences. The framework, built upon the Revit API platform and employing the RFC matching algorithm, was evaluated for both accuracy and efficiency in reflecting architects' detailing choices. The assessment involved one recipient model with objects at an LOD of 200 and three donor models with objects at LODs ranging from 350 to 400. The corresponding architects of the donor models participated in the experiment, executing two sets of detailing sequences: one using the proposed framework, and the other using conventional methods.

The findings affirm the effectiveness of the proposed BIM library transplant framework in enhancing design detailing efficiency. It potentially can reduce design detailing time by approximately 60%–70%, while achieving an accuracy of 65%–80%. Despite exhibiting some variability in performance, the framework holds significant promise for improving the current labor-intensive design detailing processes, thus affording architects more time for creative pursuits. Additionally, the authors conducted further investigations into the resulting BIM model to identify instances of inaccuracies within the current framework version and suggest areas that require future enhancements.

This study contributes to both industry and academia. From an industry perspective, it presents a method that not only streamlines labor-intensive design detailing but also aligns with architects' stylistic preferences. The application of BIM library transplants can be implemented directly in real-world scenarios, allowing architects to dedicate more time to creative exploration. Academically, this study introduces a versatile framework with modules that can be tailored to meet specific researcher needs. Because classes encompass both geometric and functional building object details, this framework's applicability extends to other BIM-based processes such as cost estimation, structural analysis, and life-cycle assessment.

However, this study has certain limitations. First, it focused predominantly on a recipient model of limited scale and three donor models with similar building occupancies. Although our findings showcase the potential of the proposed framework, further validation encompassing a broader range of scales, building occupancies, and designs from various architects and architectural firms is imperative. Second, the current framework might exhibit restricted performance when transplanting custom classes featuring sophisticated parametric rules or unconventional naming conventions, highlighting the necessity for a more adaptable extraction and transplant algorithm. Lastly, the present version supports only a one-time process without provisions for architect feedback or corrections. This could be addressed by enhancing user interfaces and introducing interactive elements to enhance usability and truly embody architects' preferences.

The inception of this study stemmed from the aspiration to create an intelligent machine capable of deciphering an architect's individual inclinations from previous projects and incorporating them into new endeavors, thereby streamlining resource-intensive design detailing. Although the current framework is in its nascent stages, enhancing the BIM library transplant framework could involve refining each module, as outlined in the "Discussion" section, through advanced classification, matching, and retrieval techniques. These efforts could leverage cutting-edge AI methodologies, such as graph- or natural-language–based approaches or user-interactive interfaces. Delving deeper into the potential applications of the framework leads toward a future in which architects can channel their efforts toward creativity, liberated from the confines of labor-intensive procedures.

Data Availability Statement

Some or all data, models, or code generated or used during the study are proprietary or confidential in nature and may be provided only with restrictions. Donor models A, B, and C used for the validation in this study were provided by the architects involved in the experiment, and therefore are proprietary in nature.

Acknowledgments

The authors thank Inyoung Park, Hwanwoong Jeong, and Yumin Park of SAAI^{+} for providing their esteemed digital assets and the generous allotment of their time for the validation of the proposed framework. This work was supported in 2024 by the Korea Agency for Infrastructure Technology Advancement (KAIA) grant funded by the Ministry of Land, Infrastructure, and Transport (Grant RS-2021-KA163269).

Notation

The following symbols are used in this paper:

B = bootstrap sample;
B_i = ith bootstrap sample;
D = original data set, composed of pairs (X, Y);
m = number of randomly selected features at each node in decision tree;
n = number of subsets or decision trees;
T = decision tree;
T_i = ith decision tree;

X = feature vector of an instance;
X_i = feature vector of ith instance;
Y = label corresponding to a feature vector;
$\hat{Y}$ = predicted class for a given instance; and
Y_i = label corresponding to ith feature vector.

Supplemental Materials

Tables S1–S24 are available online in the ASCE Library (www.ascelibrary.org).

References

Bourahla, N., S. Tafraout, Y. Bourahla, A. Sereir-El-Hirtsi, and A. Skoudarli. 2022. "GA based design automation and optimization of earthquake resisting CFS structures in a BIM environment." *Structures* 43 (Sep): 1334–1341. https://doi.org/10.1016/j.istruc.2022.07.041.

Chun, P., I. Ujike, K. Mishima, M. Kusumoto, and S. Okazaki. 2020. "Random forest-based evaluation technique for internal damage in reinforced concrete featuring multiple nondestructive testing results." *Constr. Build. Mater.* 253 (Aug): 119238. https://doi.org/10.1016/j.conbuildmat.2020.119238.

Czmoch, I., and A. Pękala. 2014. "Traditional design versus BIM based design." *Procedia Eng.* 91 (Jan): 210–215. https://doi.org/10.1016/j.proeng.2014.12.048.

Davis, J. P. 2019. "Artificial wisdom? A potential limit on AI in law (and elsewhere)." *SSRN J.* 72 (Jun): 51. https://doi.org/10.2139/ssrn.3350600.

Eldin, N. N. 1991. "Management of engineering/design phase." *J. Constr. Eng. Manage.* 117 (1): 163–175. https://doi.org/10.1061/(ASCE)0733-9364(1991)117:1(163).

Gao, G., Y.-S. Liu, M. Wang, M. Gu, and J.-H. Yong. 2015. "A query expansion method for retrieving online BIM resources based on Industry Foundation Classes." *Autom. Constr.* 56 (Jun): 14–25. https://doi.org/10.1016/j.autcon.2015.04.006.

Ghannad, P., and Y.-C. Lee. 2022. "Automated modular housing design using a module configuration algorithm and a coupled generative adversarial network (CoGAN)." *Autom. Constr.* 139 (Jul): 104234. https://doi.org/10.1016/j.autcon.2022.104234.

Giel, B. K., and R. R. A. Issa. 2013. "Return on investment analysis of using building information modeling in construction." *J. Comput. Civ. Eng.* 27 (5): 511–521. https://doi.org/10.1061/(ASCE)CP.1943-5487.0000164.

Hamidavi, T., S. Abrishami, and M. R. Hosseini. 2020. "Towards intelligent structural design of buildings: A BIM-based solution." *J. Build. Eng.* 32 (Nov): 101685. https://doi.org/10.1016/j.jobe.2020.101685.

Ho, T. K. 1995. "Random decision forests." In *Proc., 3rd Int. Conf. on Document Analysis and Recognition*, 278–282. New York: IEEE.

Hoang, N.-D., and Q.-L. Nguyen. 2019. "A novel method for asphalt pavement crack classification based on image processing and machine learning." *Eng. Comput.* 35 (2): 487–498. https://doi.org/10.1007/s00366-018-0611-9.

Hoang, N.-D., and V.-D. Tran. 2023. "Comparison of histogram-based gradient boosting classification machine, random forest, and deep convolutional neural network for pavement raveling severity classification." *Autom. Constr.* 148 (Jun): 104767. https://doi.org/10.1016/j.autcon.2023.104767.

Hosseinijou, S. A., S. Mansour, and M. A. Shirazi. 2014. "Social life cycle assessment for material selection: A case study of building materials." *Int. J. Life Cycle Assess.* 19 (3): 620–645. https://doi.org/10.1007/s11367-013-0658-1.

Jang, S., and G. Lee. 2023. "Improving BIM authoring process reproducibility with enhanced BIM logging." In *Proc., 23rd Int. Conf. on Construction Applications of Virtual Reality (CONVR) 2023*, 512–520. Florence, Italy: Firenze University Press.

Kim, J., J. Song, and J.-K. Lee. 2019. "Inference of relevant BIM objects using CNN for visual-input based auto-modeling." In *Proc., Int. Symp. on Automation and Robotics in Construction*, 393–398. Edinburgh, UK: International Association for Automation and Robotics in Construction Publications. https://doi.org/10.22260/ISARC2019/0053.

Koo, B., S. La, N.-W. Cho, and Y. Yu. 2019. "Using support vector machines to classify building elements for checking the semantic integrity of building information models." *Autom. Constr.* 98 (Feb): 183–194. https://doi.org/10.1016/j.autcon.2018.11.015.

Koo, B., B. Shin, and G. Lee. 2017. "A cost-plus estimating framework for BIM related design and engineering services." *KSCE J. Civ. Eng.* 21 (7): 2558–2566. https://doi.org/10.1007/s12205-017-1808-y.

Kotsiantis, S. B., I. Zaharakis, and P. Pintelas. 2007. "Supervised machine learning: A review of classification techniques." In *Emerging artificial intelligence applications in computer engineering*, 3–24. Amsterdam, Netherlands: IOS Press.

Lee, G., R. Sacks, and C. M. Eastman. 2006. "Specifying parametric building object behavior (BOB) for a building information modeling system." *Autom. Constr.* 15 (6): 758–776. https://doi.org/10.1016/j.autcon.2005.09.009.

Leite, F., A. Akcamete, B. Akinci, G. Atasoy, and S. Kiziltas. 2011. "Analysis of modeling effort and impact of different levels of detail in building information models." *Autom. Constr.* 20 (5): 601–609. https://doi.org/10.1016/j.autcon.2010.11.027.

Li, N., Q. Li, Y.-S. Liu, W. Lu, and W. Wang. 2020. "BIMSeek++: Retrieving BIM components using similarity measurement of attributes." *Comput. Ind.* 116 (Jun): 103186. https://doi.org/10.1016/j.compind.2020.103186.

Lin, B., H. Chen, Q. Yu, X. Zhou, S. Lv, Q. He, and Z. Li. 2021. "MOOSAS—A systematic solution for multiple objective building performance optimization in the early design stage." *Build. Environ.* 200 (Aug): 107929. https://doi.org/10.1016/j.buildenv.2021.107929.

Liu, H., G. Singh, M. Lu, A. Bouferguene, and M. Al-Hussein. 2018. "BIM-based automated design and planning for boarding of light-frame residential buildings." *Autom. Constr.* 89 (May): 235–249. https://doi.org/10.1016/j.autcon.2018.02.001.

Lu, S. C.-Y., and A. Liu. 2011. "Subjectivity and objectivity in design decisions." *CIRP Ann.* 60 (1): 161–164. https://doi.org/10.1016/j.cirp.2011.03.122.

Luo, Z., and W. Huang. 2022. "FloorplanGAN: Vector residential floorplan adversarial generation." *Autom. Constr.* 142 (Oct): 104470. https://doi.org/10.1016/j.autcon.2022.104470.

Mangal, M., and J. C. P. Cheng. 2018. "Automated optimization of steel reinforcement in RC building frames using building information modeling and hybrid genetic algorithm." *Autom. Constr.* 90 (Jan): 39–57. https://doi.org/10.1016/j.autcon.2018.01.013.

Mathew, A., P. Amudha, and S. Sivakumari. 2021. "Deep learning techniques: An overview." In *Proc., AMLTA 2020 Advanced Machine Learning Technologies and Applications*, 599–608. Berlin: Springer. https://doi.org/10.1007/978-981-15-3383-9_54.

Noguerol, T. M., F. Paulano-Godino, M. T. Martín-Valdivia, C. O. Menias, and A. Luna. 2019. "Strengths, weaknesses, opportunities, and threats analysis of artificial intelligence and machine learning applications in radiology." *J. Am. Coll. Radiol.* 16 (9): 1239–1247. https://doi.org/10.1016/j.jacr.2019.05.047.

Pan, Y., and L. Zhang. 2023. "Integrating BIM and AI for smart construction management: Current status and future directions." *Arch. Comput. Methods Eng.* 30 (2): 1081–1110. https://doi.org/10.1007/s11831-022-09830-8.

Phillipson, M. C., R. Emmanuel, and P. H. Baker. 2016. "The durability of building materials under a changing climate." *WIREs Clim. Change* 7 (4): 590–599. https://doi.org/10.1002/wcc.398.

Qian, W., Y. Xu, and H. Li. 2022. "A self-sparse generative adversarial network for autonomous early-stage design of architectural sketches." *Comput.-Aided Civ. Infrastruct. Eng.* 37 (5): 612–628. https://doi.org/10.1111/mice.12759.

Senior, M. L., C. J. Webster, and N. E. Blank. 2006. "Residential relocation and sustainable urban form: Statistical analyses of owner-occupiers' preferences." *Int. Plan. Stud.* 11 (1): 41–57. https://doi.org/10.1080/13563470600935024.

Sun, C., Y. Zhou, and Y. Han. 2022. "Automatic generation of architecture facade for historical urban renovation using generative adversarial network." *Build. Environ.* 212 (Mar): 108781. https://doi.org/10.1016/j.buildenv.2022.108781.

Tafraout, S., N. Bourahla, Y. Bourahla, and A. Mebarki. 2019. "Automatic structural design of RC wall-slab buildings using a genetic algorithm with application in BIM environment." *Autom. Constr.* 106 (Oct): 102901. https://doi.org/10.1016/j.autcon.2019.102901.

Wu, S., Q. Shen, Y. Deng, and J. Cheng. 2019. "Natural-language-based intelligent retrieval engine for BIM object database." *Comput. Ind.* 108 (Jun): 73–88. https://doi.org/10.1016/j.compind.2019.02.016.

Wu, S., N. Zhang, X. Luo, and W.-Z. Lu. 2021. "Intelligent optimal design of floor tiles: A goal-oriented approach based on BIM and parametric design platform." *J. Cleaner Prod.* 299 (May): 126754. https://doi.org/10.1016/j.jclepro.2021.126754.

Yan, X., D. Bao, Y. Zhou, Y. Xie, and T. Cui. 2022. "Detail control strategies for topology optimization in architectural design and development." *Front. Archit. Res.* 11 (2): 340–356. https://doi.org/10.1016/j.foar.2021.11.001.

Ye, X., J. Du, and Y. Ye. 2022. "MasterplanGAN: Facilitating the smart rendering of urban master plans via generative adversarial networks." *Environ. Plan. B: Urban Anal. City Sci.* 49 (3): 794–814. https://doi.org/10.1177/23998083211023516.

Yenerim, D., and W. Yan. 2011. "BIM-based parametric modeling: A case study." In *Proc., Int. Conf. on Modeling, Simulation and Visualization Methods (MSV)*. University Park, PA: Citeseer.

Zambrano, J. M., U. Filippi Oberegger, and G. Salvalai. 2021. "Towards integrating occupant behaviour modelling in simulation-aided building design: Reasons, challenges and solutions." *Energy Build.* 253 (Jun): 111498. https://doi.org/10.1016/j.enbuild.2021.111498.

Zhang, S., Z. Han, Y.-K. Lai, M. Zwicker, and H. Zhang. 2019. "Stylistic scene enhancement GAN: Mixed stylistic enhancement generation for 3D indoor scenes." *Vis. Comput.* 35 (6–8): 1157–1169. https://doi.org/10.1007/s00371-019-01691-w.